

Terraform Task 4

Why Terraform is important

1. **Infrastructure as Code (IaC)**
Write infrastructure in .tf files → version-controlled, repeatable, and auditable.
2. **Automation & Speed**
Create servers, networks, databases, and services in minutes with init → plan → apply.
3. **Consistency & No Human Error**
Same code = same infrastructure across dev, test, and prod.
4. **Multi-Cloud Support**
Works with AWS, Azure, GCP, and on-prem from a **single tool**.
5. **State Management**
Tracks real infrastructure using a **state file**, enabling safe updates and drift detection.
6. **Idempotency**
Running Terraform multiple times won't break infra — it only changes what's needed.
7. **Easy CI/CD Integration**
Integrates smoothly with Jenkins, GitHub Actions, GitLab CI, etc.
8. **Scalable & Modular**
Reuse infrastructure using **modules** as your environment grows.
9. **Cost & Resource Control**
Plan before apply → know exactly what will be created or destroyed.
10. **Industry Standard**
Widely used in DevOps and cloud roles; strong interview and production relevance.

One-line interview answer

Terraform is important because it automates infrastructure provisioning using code, ensuring consistency, scalability, and multi-cloud support with safe, repeatable deployments.

1. Watch Terraform-04 video.
2. Execute the script shown in the video.

- Go to terraform site and then hasicorp local the select version
- You will find terraform script

The screenshot shows the HashiCorp Terraform Registry page for the 'local' provider, version 2.6.0. The page includes the HashiCorp logo, the provider name 'local', and a description: 'Used to manage local resources, such as creating files'. It also shows the version '2.6.0' and the publication date '2 months ago'. A 'USE PROVIDER' button is visible. On the right, there is a section titled 'How to use this provider' with instructions on how to install the provider and a code snippet for the Terraform configuration.

```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "2.6.0"
    }
  }
}

provider "local" {
  # Configuration options
}
```

- Paste the script in the cli terraform main.tf

The screenshot shows a VS Code editor with the Terraform configuration in the 'main.tf' file. The configuration defines the 'local' provider and a resource 'local_file' named 'my_pet'. The terminal output shows the execution of 'terraform init', which successfully initializes the workspace by downloading the 'local' provider and creating a lock file. The output also includes instructions on how to use Terraform.

```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "2.6.0"
6     }
7   }
8 }
9
10
11
12 resource "local_file" "my_pet" {
13   filename = "pets.txt"
14   content = "My cat name is ${random_pet.petname.id}"
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
- Finding latest version of hashicorp/random...
- Finding hashicorp/local versions matching "2.6.0"...
- Installing hashicorp/random v3.8.0...
- Installed hashicorp/random v3.8.0 (signed by HashiCorp)
- Installing hashicorp/local v2.6.0...
- Installed hashicorp/local v2.6.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basics>
```

Version Constraints:

=====

Changing in terraform providers version may get us into incompatibility issues.

By default terraform will always try to download the latest version of provider available from registry.

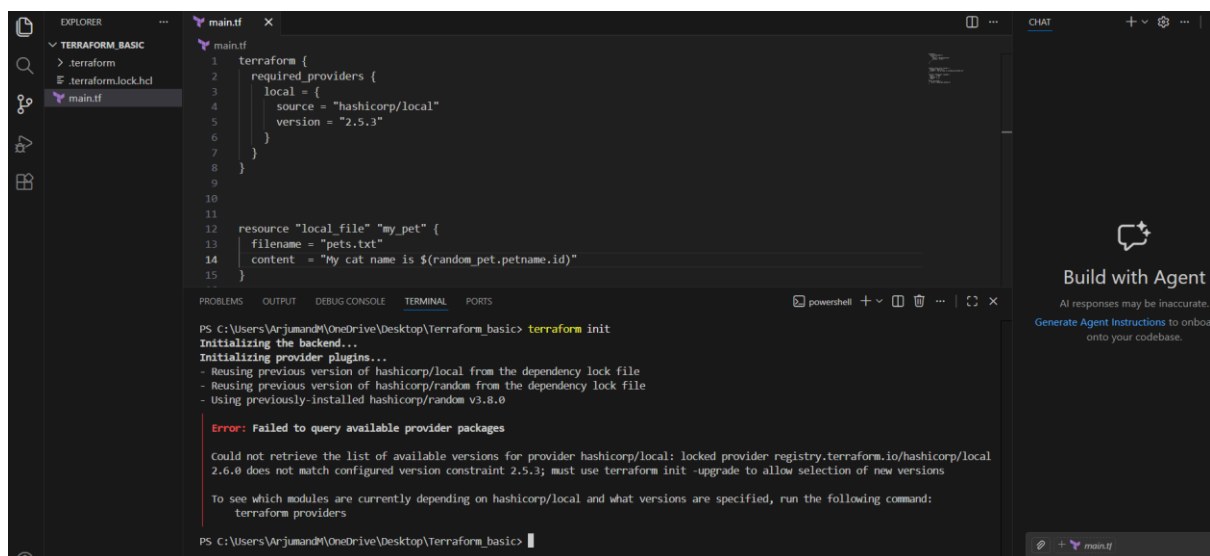
To make sure to use the specific version provider we can add the provider block in configuration.

Example:

=====

```
terraform {  
  required_providers {  
    local = {  
      source = "hashicorp/local"  
      version = "2.3.0"  
    }  
  }  
}
```

- Then terraform init and then apply terraform apply



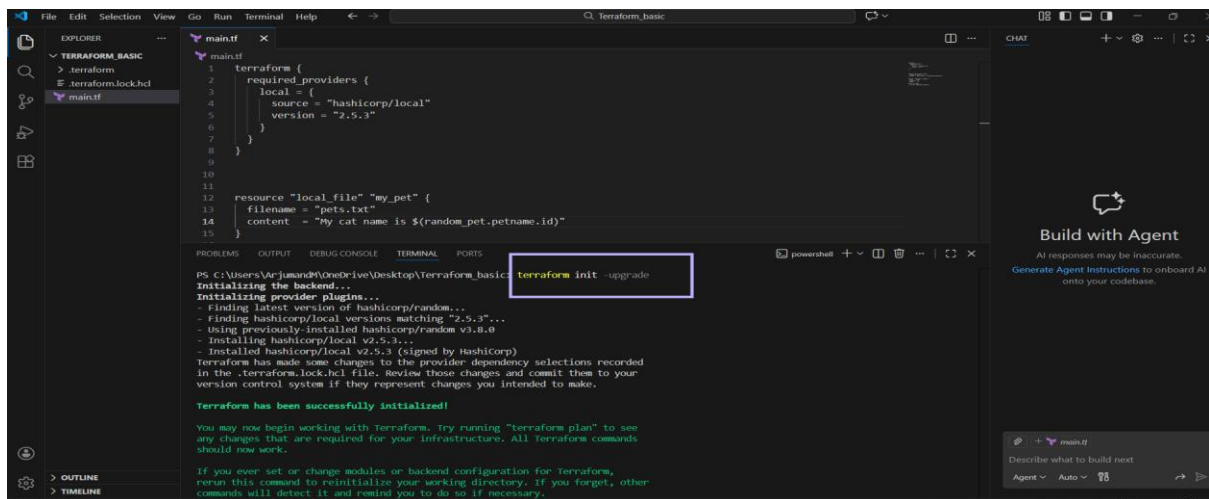
version = "2.3.0" --> download the exact version

version = "!=2.3.0" --> will not use the mentioned version

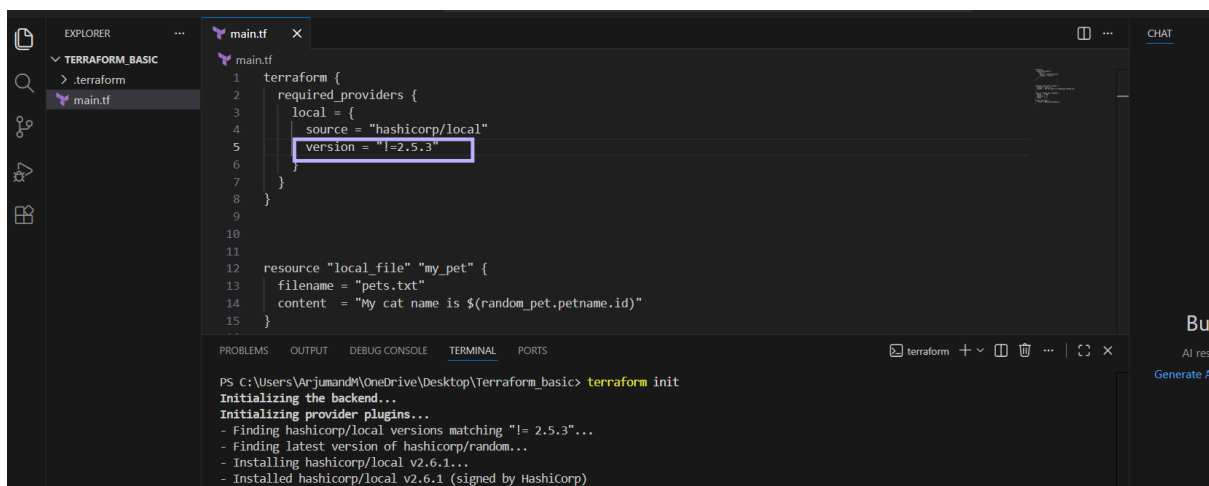
version = "< 2.3.0" --> less than the mentioned version

version = "> 2.3.0" --> greater than the given version

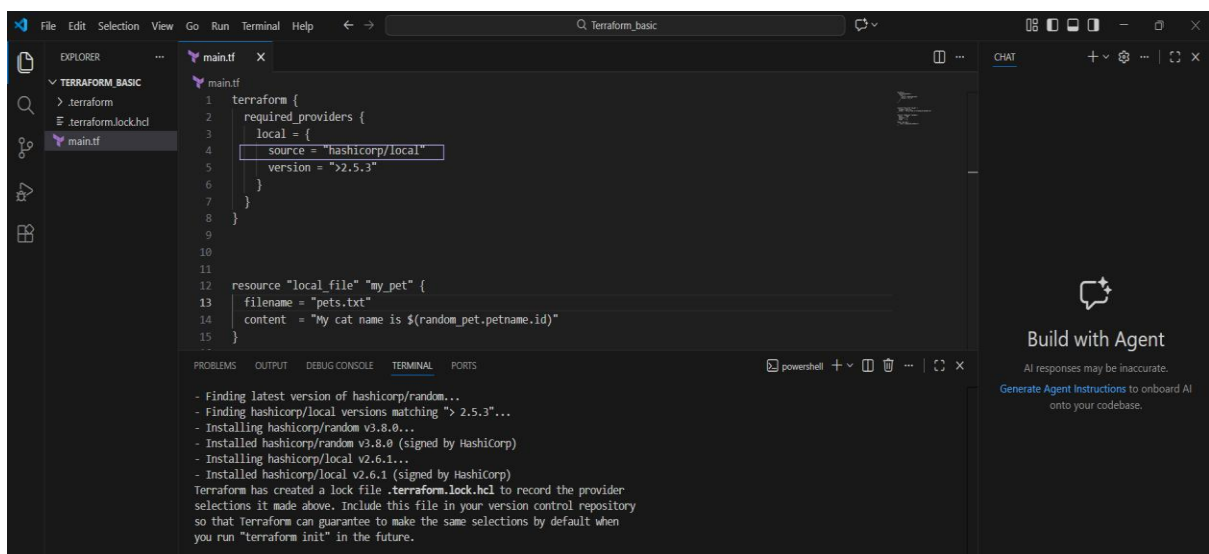
version = "~> 2.3.0" --> specific version or higher version.



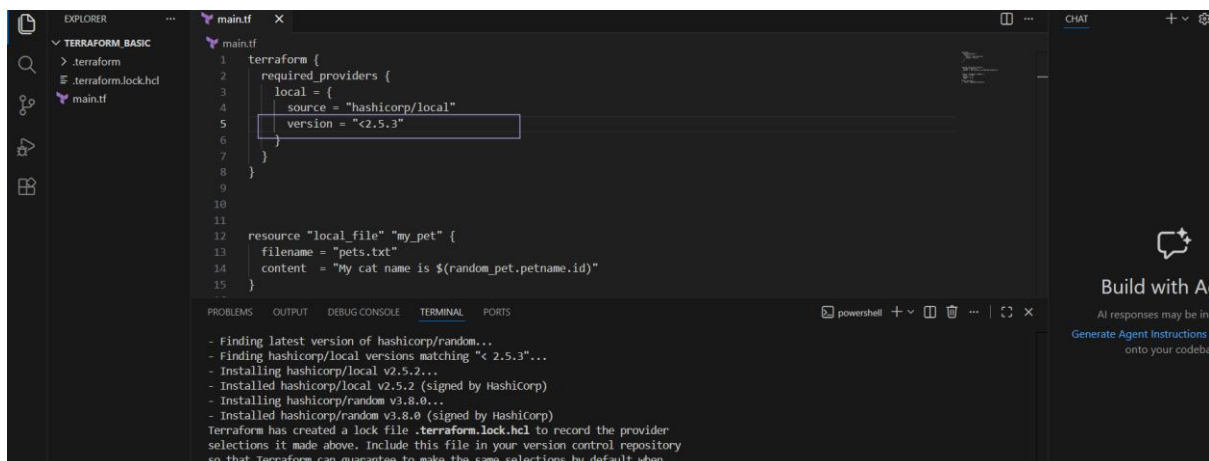
version = "!2.3.0" --> will not use the men@oned version



version = "> 2.3.0" --> greater than the given version



version = "< 2.3.0" --> specific version or higher version.

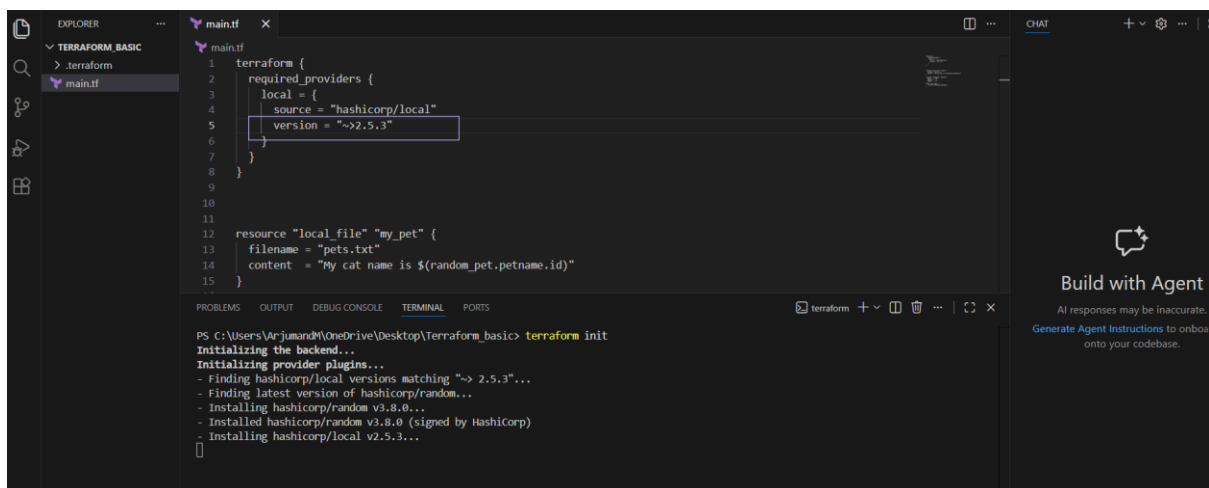


```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "<2.5.3"
6     }
7   }
8 }
9
10
11
12 resource "local_file" "my_pet" {
13   filename = "pets.txt"
14   content = "My cat name is ${random_pet.petname.id}"
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
- Finding latest version of hashicorp/random...
- Finding hashicorp/local versions matching "< 2.5.3"...
- Installing hashicorp/local v2.5.2...
- Installed hashicorp/local v2.5.2 (signed by HashiCorp)
- Installing hashicorp/random v3.8.0...
- Installed hashicorp/random v3.8.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
```

version = "~> 2.3.0" --> specific version or higher version.



```
1 terraform {
2   required_providers {
3     local = {
4       source = "hashicorp/local"
5       version = "~>2.5.3"
6     }
7   }
8 }
9
10
11
12 resource "local_file" "my_pet" {
13   filename = "pets.txt"
14   content = "My cat name is ${random_pet.petname.id}"
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Arjun\OneDrive\Desktop\Terraform_basics> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.5.3"...
- Finding latest version of hashicorp/random...
- Installing hashicorp/random v3.8.0...
- Installed hashicorp/random v3.8.0 (signed by HashiCorp)
- Installing hashicorp/local v2.5.3...
```

Data sources:

=====

Apart from terraform we have multiple other tools where the infra can be created.

Ex: ansible,salt,puppet,bash script>manual process.

Data sources are used to read the content of the infrastructure

for example if we want terraform to read the content of the file which has been created by any other tool.

create a file called in dogs.txt in the same terraform working directory.

main.tf

=====

```
resource "local_file" "my-pet" {
  filename = "pets.txt"
```

```

content = data.local_file.dog.content
}
data "local_file" "dog" {
filename = "dogs.txt"

```

```

2 resource "local_file" "my_pet" {
3   filename = "pets.txt"
4   content  = data.local_file.dog.content
5 }
6
7 data "local_file" "dog" {
8   filename = "dog.txt"
9 }
10

```

```

+ content_sha256      = (known after apply)
+ content_sha512      = (known after apply)
+ directory_permission = "0777"
+ file_permission     = "0777"
+ filename            = "pets.txt"
+ id                  = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=4e7c940deabb5648c06874f3d330df05f97a24]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basi>

```

- Provide variable values in variable.tf file also add count as 3

```

1 resource "local_file" "my_pet" {
2   filename = var.filename[count.index]
3   content  = "I love animals"
4   count    = 3
5 }
6
7

```

```

1 variable "filename" {
2   default = [
3     "pet.txt",
4     "cat.txt",
5     "Rabbit.txt"
6   ]
7 }

```

```

+ filename            = "Rabbit.txt"
+ id                  = (known after apply)
}

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.my_pet[2]: Creating...
local_file.my_pet[1]: Creating...
local_file.my_pet[0]: Creating...
local_file.my_pet[0]: Creation complete after 0s [id=253e577f04b608fb1574769e1f61e7988ec93fc]
local_file.my_pet[2]: Creation complete after 0s [id=253e577f04b608fb1574769e1f61e7988ec93fc]
local_file.my_pet[1]: Creation complete after 0s [id=253e577f04b608fb1574769e1f61e7988ec93fc]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basi>

```

- Meta-Arguments:

=====

Meta arguments are used if we want to create multiple resources.

Meta arguments can be used within any resource block to change the behaviour of the resources.

Examples for meta arguments:

- 1) Depends_on
- 2) lifecycle rules
- 3) Count
- 4) For_each

Example of count:

=====

- Add count as length (var.filename)

The screenshot shows a VS Code editor with the following files open: `main.tf`, `variable.tf`, and `main.tf`. The `main.tf` file contains the following Terraform configuration:

```
1 resource "local_file" "my_pet" {
2   filename = var.filename[count.index]
3   content  = "I love animals"
4   count    = length(var.filename)
5 }
6
7
8
```

The `variable.tf` file contains the following Terraform configuration:

```
1 variable "filename" {
2   default = [
3     "Tiger.txt",
4     "cat.txt",
5     "Rabbit.txt"
6   ]
7 }
```

The terminal window shows the output of the Terraform command:

```
Plan: 3 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet[1]: Destroying... [id=253e577f04b608fb1574769ee1f61e7988ec93fc]
local_file.my_pet[0]: Destroying... [id=253e577f04b608fb1574769ee1f61e7988ec93fc]
local_file.my_pet[0]: Destruction complete after 0s
local_file.my_pet[1]: Destruction complete after 0s
local_file.my_pet[2]: Creating...
local_file.my_pet[2]: Creation complete after 0s [id=253e577f04b608fb1574769ee1f61e7988ec93fc]
local_file.my_pet[0]: Creating...
local_file.my_pet[1]: Creating...
local_file.my_pet[1]: Creation complete after 0s [id=253e577f04b608fb1574769ee1f61e7988ec93fc]
local_file.my_pet[0]: Creation complete after 0s [id=253e577f04b608fb1574769ee1f61e7988ec93fc]

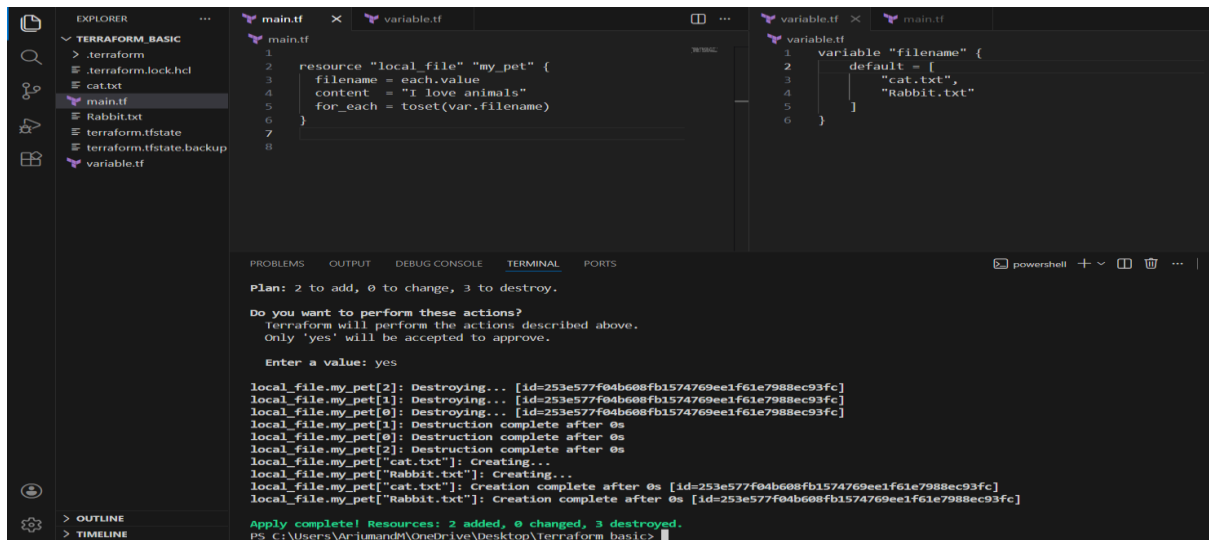
Apply complete! Resources: 3 added, 0 changed, 2 destroyed.
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basics>
```

Example of for_each:

Filename = each.value

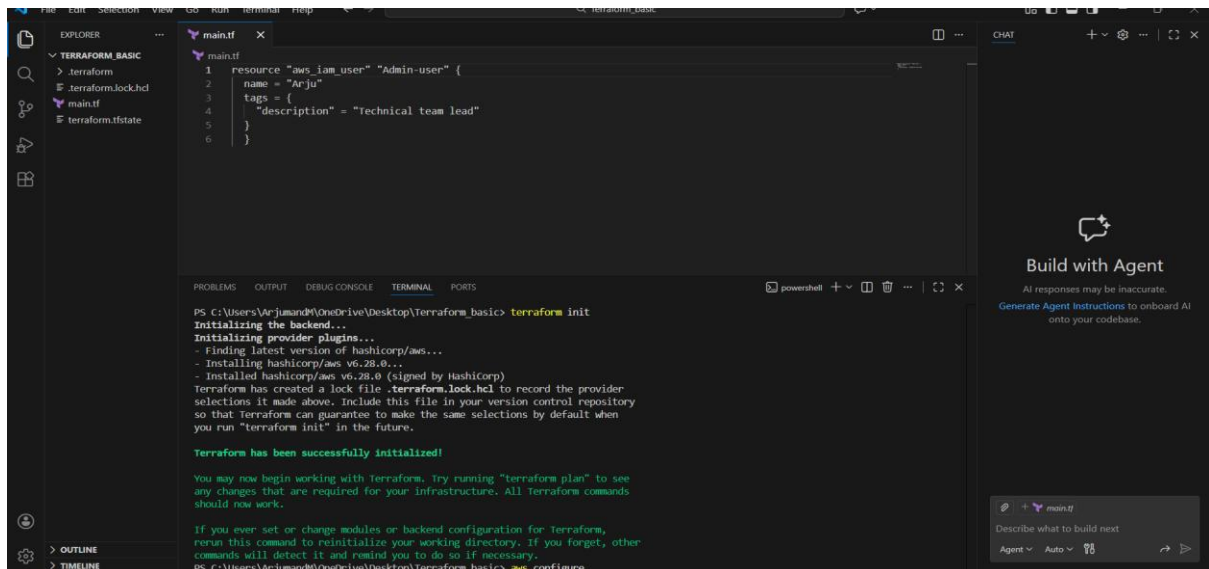
Provide value in each = toset(var.filename)

- After changing the script save it
- Enter terraform apply in cli
- You can modify the number of files and don't need to manually change the count of the file, it set automatically and support list data as well



AWS IAM USER:

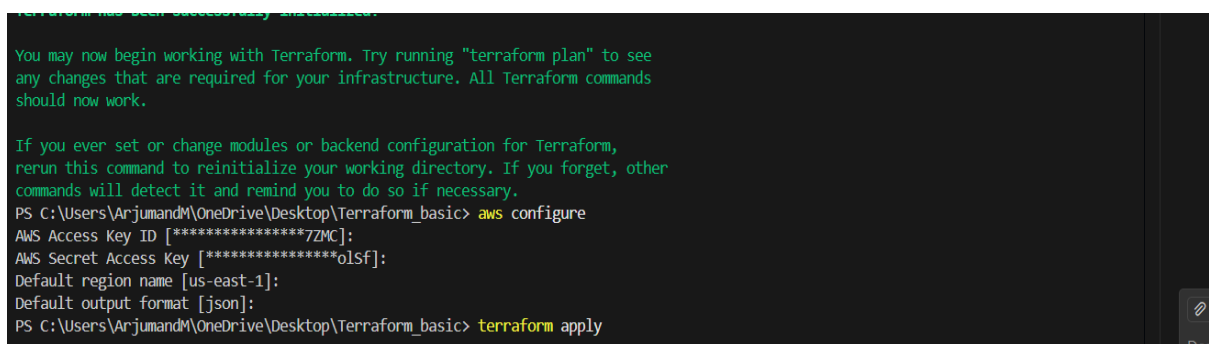
- Simple IAM user script as shown in the below main.tf file



Terraform with AWS:

=====

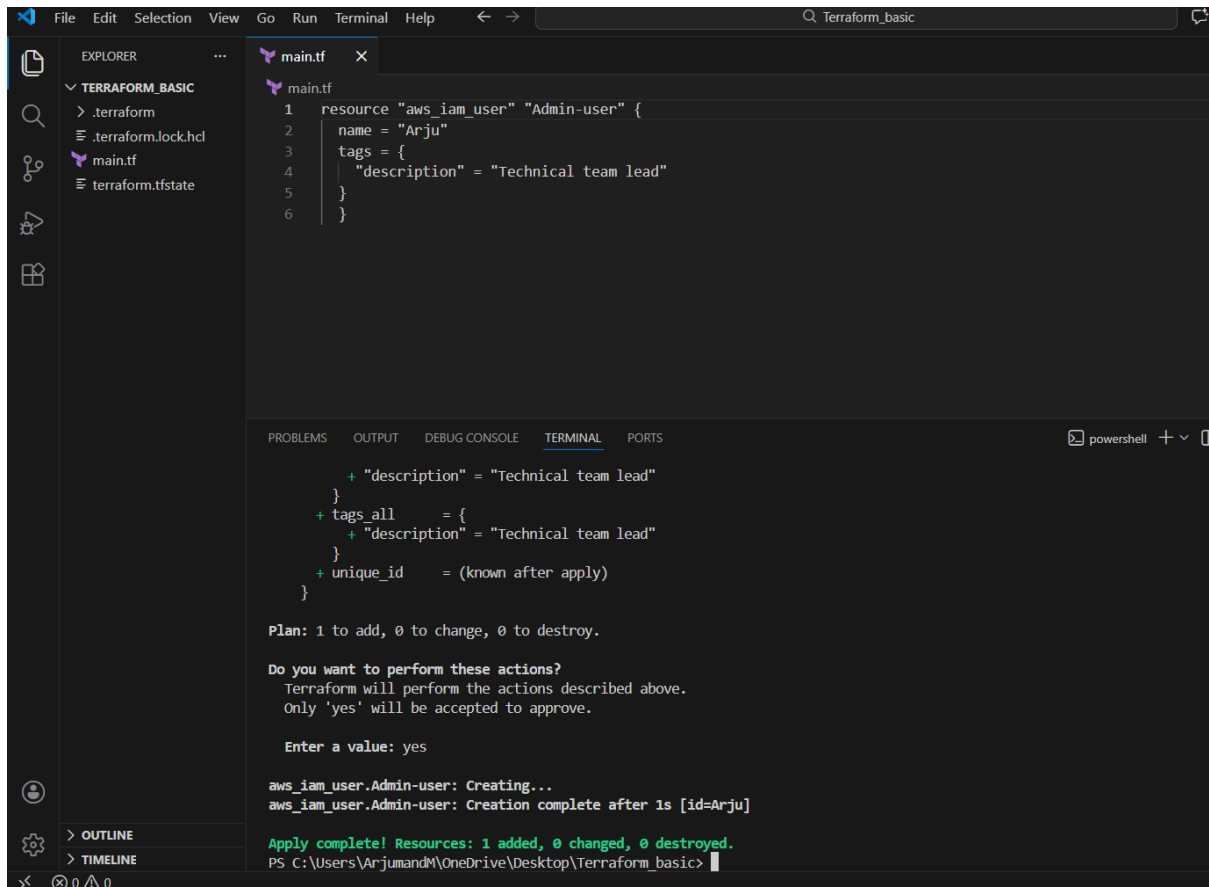
--> first we need to create a secret key and access key and configure then in in CLI, using AWS configure



I have provided a user called 'Arju'

sample script to create aws iam:

```
=====
resource "aws_iam_user" "Admin-user" {
  name = "Arju"
  tags = {
    "description" = "Technical Team Lead"
  }
}
```



The screenshot shows the Visual Studio Code editor with a file named `main.tf` containing a Terraform resource definition for an AWS IAM user. The terminal window at the bottom shows the output of a Terraform command, indicating that the user 'Arju' was created successfully.

```
main.tf
1 resource "aws_iam_user" "Admin-user" {
2   name = "Arju"
3   tags = {
4     "description" = "Technical team lead"
5   }
6 }
```

Terminal Output:

```
+ "description" = "Technical team lead"
}
+ tags_all      = {
+   "description" = "Technical team lead"
}
+ unique_id     = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

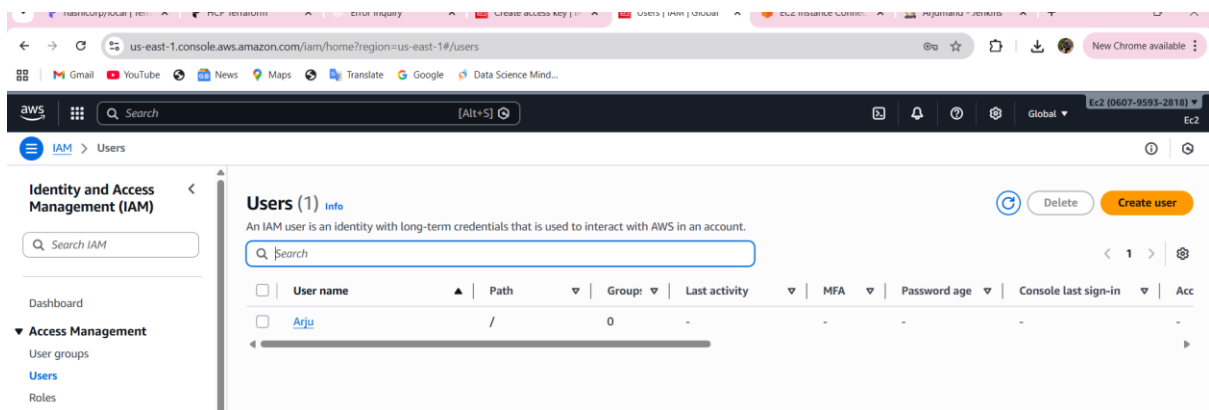
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_iam_user.Admin-user: Creating...
aws_iam_user.Admin-user: Creation complete after 1s [id=Arju]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic>
```

- User is created successfully as we can confirm with AWS IAM console



```
resource "aws_iam_user" "Admin-user" {
```

```

name = "Arju"

tags = {

"description" = "Technical Team Lead"

}

resource "aws_iam_policy" "adminuser" {

name = "AdminUsers"

policy = <<EOF

{

"Version": "2012-10-17",

"Statement": [

{

"Sid": "1234567890",

"Effect": "Allow",

"Action": "*",

"Resource": "*"

}

]

}

EOF

}

resource "aws_iam_user_policy_attachment" "Arju-admin-access" {

user = aws_iam_user.Admin-user.name

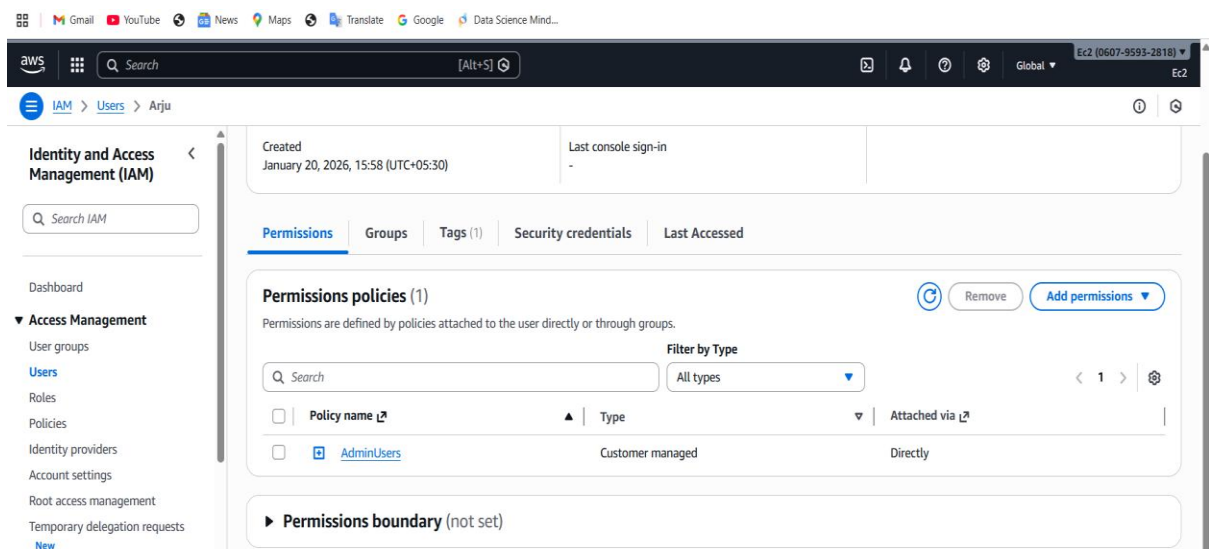
policy_arn = aws_iam_policy.adminuser.arn

}

```

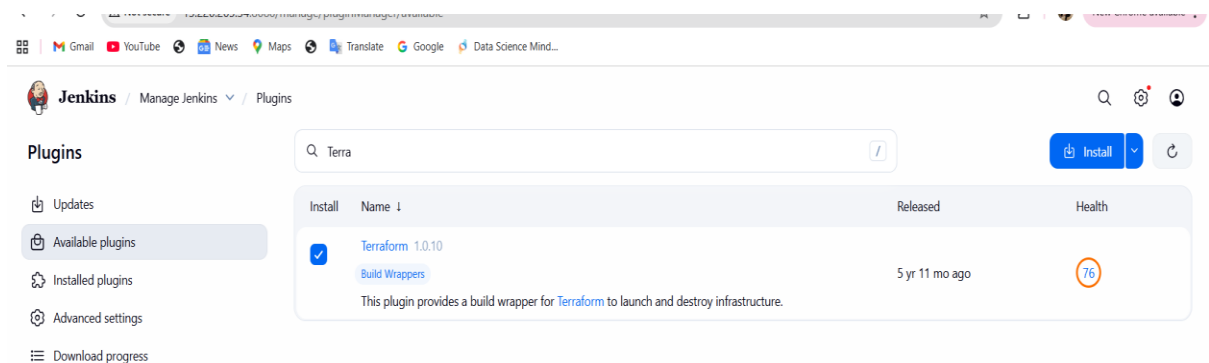
- We will providing script of the policy to the user

- Policy is successfully attached after terraform apply



3. Integrate Terraform in Jenkins using the Terraform plugin.

- Go to Jenkins server > plugin > available plugin > install plugin 'Terraform'



- Add terraform plugin in Global tools

4. Create a CI/CD pipeline for a Nodejs

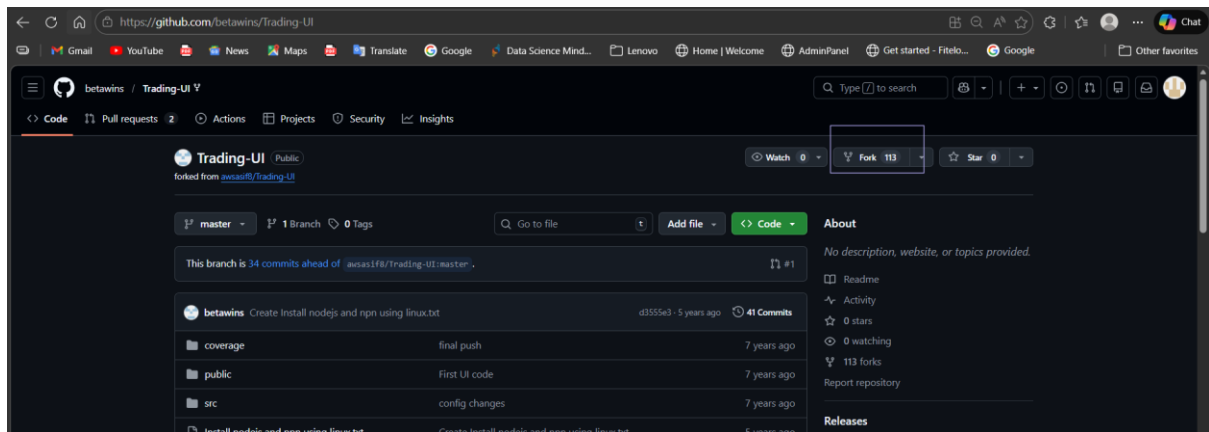
Application: <https://github.com/betawins/Trading-UI.git>

- Install nodejs plugin go to Jenkins GUI > manage plugin > Available plugin > install plugin

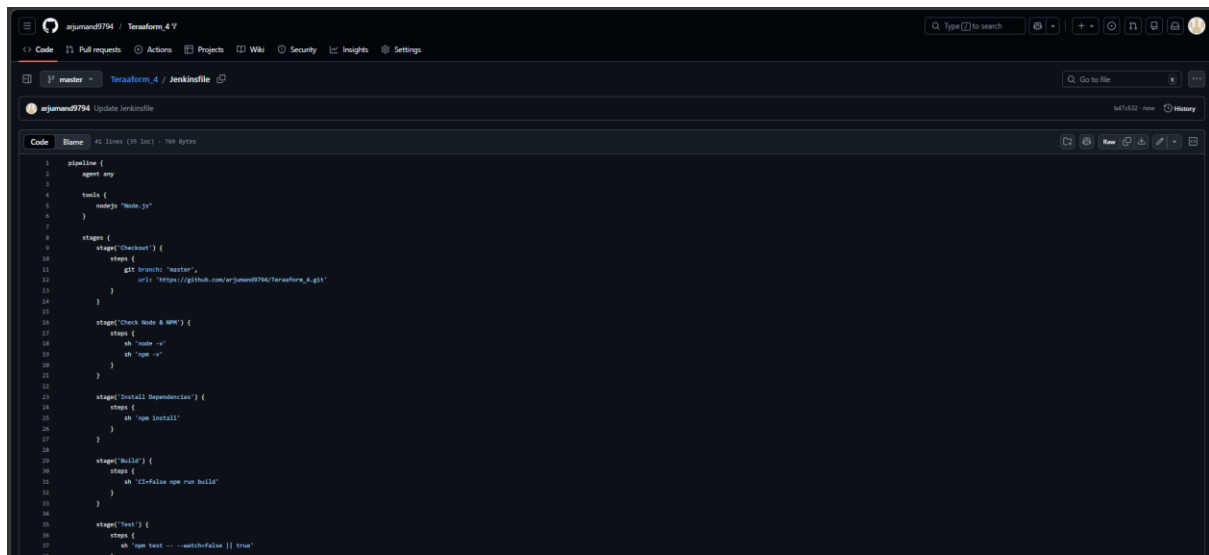
Install	Name	Released	Health
☐	Role-based Authorization Strategy 848.va_a_ea_673cf0b_c Security Authentication and User Management Enables user authorization using a Role-Based strategy. Roles can be defined globally or for particular jobs or nodes selected by regular expressions.	1 mo 18 days ago	100
☒	NodeJS 1.6.6 npm NodeJS Plugin executes NodeJS script as a build step.	2 mo 6 days ago	100
☐	Throttle Concurrent Builds 625.vc8b_e469e9a_b_c Build Wrappers Cluster Management Agent Management This plugin allows for throttling the number of concurrent builds of a project running per node or	3 days 19 hr ago	100

- Add Nodejs plugin in Global tool and select version which should not be latest

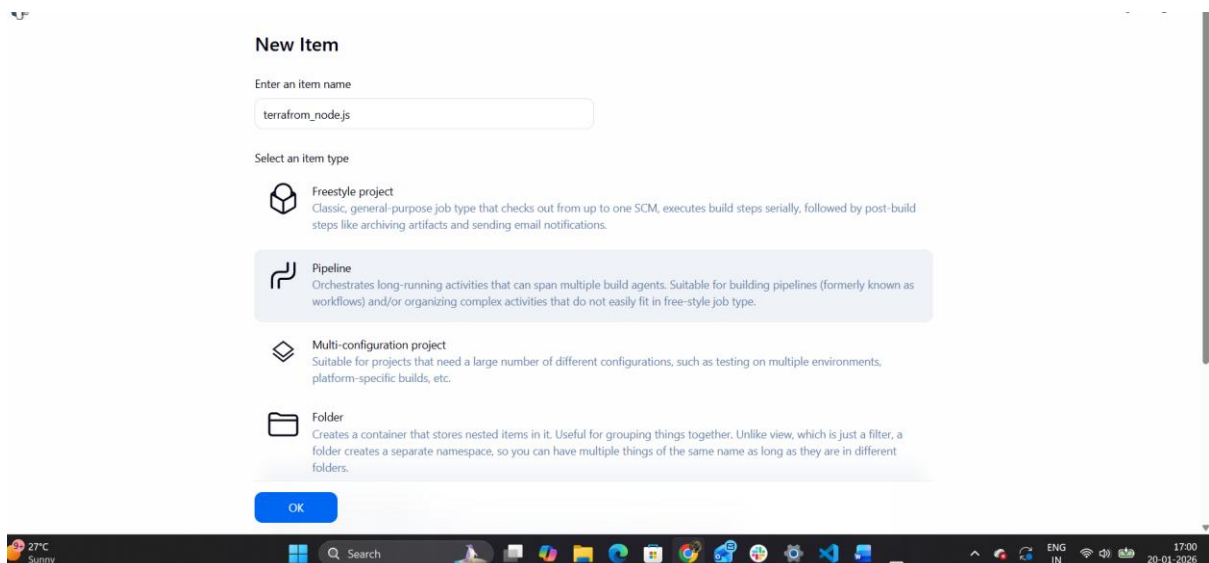
- Fork the repo from the given URL of the repo to our repo



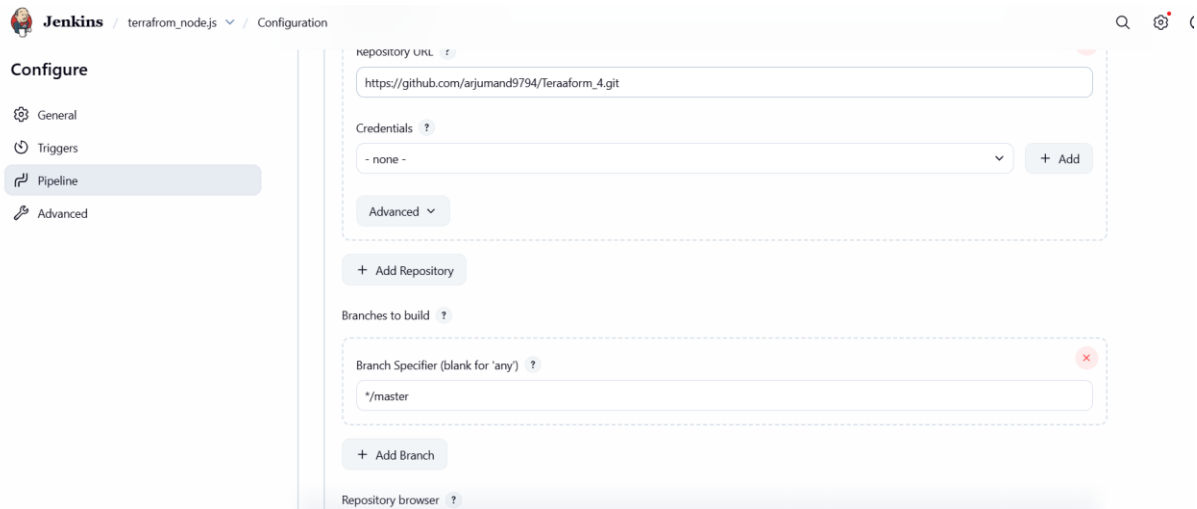
- Under our forked repo > in jenkinsfile > add terraform script to execute



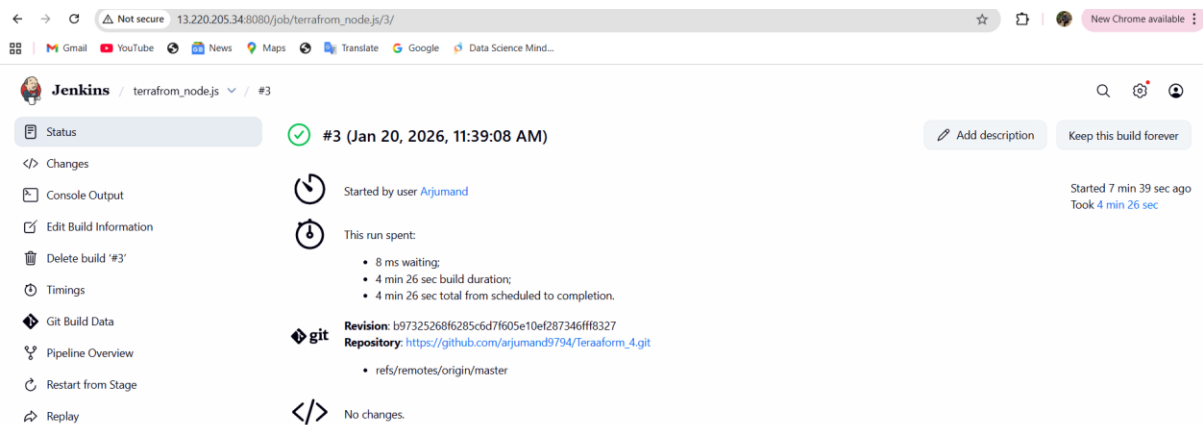
- Create a new pipeline job
- And select repo and select scm



- Select branch and our URL
- And add select script path as Jenkinsfile



- Save and build the job, job will be executed



5. Explain 10 Maven commands.

✓ 1. mvn clean

Purpose:

Deletes the previous build files.

What it does:

- Removes the target/ folder

Why used:

To ensure a fresh build.

📌 **Example:**

mvn clean

✅ 2. mvn compile

Purpose:

Compiles the source code.

What it does:

- Compiles Java files
- Checks syntax errors
- Does NOT create JAR/WAR

📌 **Example:**

mvn compile

✅ 3. mvn test

Purpose:

Runs unit test cases.

What it does:

- Executes JUnit / TestNG tests
- Stops build if tests fail

📌 **Example:**

mvn test

✅ 4. mvn package

Purpose:

Creates deployable file.

What it does:

- Compiles code
- Runs tests
- Creates:
 - .jar → standalone app
 - .war → web app

📌 Example:

`mvn package`

✅ 5. mvn install

Purpose:

Installs project into local repository.

What it does:

- Builds project
- Stores artifact in:
- `~/.m2/repository`

Used when:

- Another project depends on this one

📌 Example:

`mvn install`

✅ 6. mvn deploy

Purpose:

Deploys artifact to remote repository.

What it does:

- Uploads JAR/WAR to:
 - Nexus
 - Artifactory

Used in CI/CD pipelines.

📌 Example:

`mvn deploy`

✅ 7. mvn validate

Purpose:

Validates project structure.

Checks:

- pom.xml correctness
- Required directories

📌 Example:

`mvn validate`

✅ 8. mvn clean install

Purpose:

Most commonly used command.

What it does:

- Deletes old build
- Builds new artifact
- Installs locally

📌 Example:

`mvn clean install`

★ Used heavily in Jenkins pipelines.

✅ 9. mvn dependency:tree

Purpose:

Shows dependency hierarchy.

Used for:

- Debug dependency conflicts
- Version mismatch issues

📌 Example:

`mvn dependency:tree`

✅ 10. mvn help:effective-pom

Purpose:

Shows final merged POM.

What it shows:

- Parent POM
- Child POM
- Plugin defaults

Used when troubleshooting builds.

📌 Example:

`mvn help:effective-pom`

🔥 MOST IMPORTANT FOR INTERVIEWS

★ Most commonly used Maven commands:

Command	Usage
<code>mvn clean install</code>	Daily CI/CD
<code>mvn package</code>	Create artifact
<code>mvn test</code>	Unit testing
<code>mvn deploy</code>	Push to Nexus
<code>mvn compile</code>	Compile code

🧠 Interview one-line answer

Maven is a build automation tool used to build, test, package, and manage dependencies of Java applications using pom.xml.
